Informe Práctica 5A: Pruebas de Software

1. Introducción

En esta práctica se han realizado pruebas sobre la aplicación de seguros desarrollada en prácticas anteriores. En concreto, se ha partido de la versión final mejorada de la Práctica 3 y se ha copiado el proyecto en la carpeta correspondiente a la Práctica 5A, tal y como indica el enunciado.

El objetivo principal de la práctica ha sido diseñar y ejecutar pruebas unitarias y pruebas de integración. Para las pruebas unitarias se han probado las clases del dominio Cliente y Seguro, sin probar métodos get/set. Para las pruebas de integración se ha probado la clase VistaAgente, comprobando su funcionamiento junto con la capa de negocio y la capa DAO. Además, se ha utilizado JUnit para las pruebas y AssertJ Swing para las pruebas de interfaz gráfica.

2. Preparación del proyecto

Para realizar esta práctica he creado una carpeta nueva llamada Practica5A y he copiado dentro el contenido de la carpeta Practica3Final, que era la versión mejorada obtenida en la Práctica 3.

Después he abierto la carpeta Practica5A en Eclipse como proyecto Maven. También he comprobado desde terminal que el proyecto compilaba correctamente ejecutando:

mvn clean verify

El resultado fue correcto, por lo que pude empezar a añadir las pruebas.

3. Pruebas unitarias de las clases del dominio

La primera parte de la práctica consiste en probar las clases del dominio Cliente y Seguro. No se han probado los getters y setters, ya que no tienen lógica propia y el enunciado indica que deben excluirse.

Las pruebas unitarias se han creado en el módulo seguros-common, dentro de src/test/java.

Para poder ejecutar las pruebas unitarias he añadido JUnit al pom.xml de seguros-common.

3.1 Pruebas unitarias de la clase Cliente

Método probado

En la clase Cliente el método que contiene lógica es: *public double totalSeguros()*

Este método calcula el total que tiene que pagar un cliente por todos sus seguros. Si el cliente tiene minusvalía, se aplica un descuento del 25%.

Diseño de casos de prueba

Para diseñar las pruebas he usado principalmente caja negra, pensando en los distintos casos posibles de entrada:

Caso	Situación probada	Resultado esperado
1	Cliente sin seguros	Total 0
2	Lista de seguros null	Total 0
3	Cliente con seguros y sin minusvalía	Suma normal de precios
4	Cliente con seguros y con minusvalía	Suma con descuento del 25%
5	Lista con un seguro null	Ignorar el seguro null

Clase de prueba

La clase de prueba creada ha sido: *ClienteTest*

En esta clase se han creado distintos objetos Cliente y Seguro para comprobar el comportamiento de totalSeguros().

Los tests implementados fueron:

- *totalSegurosDevuelveCeroSiNoHaySeguros*
- *totalSegurosDevuelveCeroSiLaListaEsNull*
- *totalSegurosSumaPreciosSinMinusvalia*
- *totalSegurosAplicaDescuentoSiHayMinusvalia*
- *totalSegurosIgnoraSegurosNull*

Resultado

Al ejecutar: *mvn clean test*

los tests de ClienteTest pasaron correctamente.

También ejecuté la cobertura con EclEmma usando: *Coverage As > JUnit Test*

y el método totalSeguros() quedó cubierto en verde.

3.2 Pruebas unitarias de la clase Seguro

Método probado

En la clase Seguro el método principal con lógica es: *public double precio()*

Este método calcula el precio de un seguro teniendo en cuenta:

- la cobertura;
- la potencia;
- la fecha de inicio;
- si el seguro todavía no ha comenzado;
- si está dentro del primer año y se aplica una oferta.

Diseño de casos de prueba

Para esta clase se han usado particiones equivalentes, ya que el precio cambia según distintos grupos de valores.

Caso	Situación probada	Resultado esperado
1	Fecha de inicio null	Precio 0
2	Fecha de inicio futura	Precio 0
3	Cobertura TERCEROS, potencia baja, sin oferta	Precio 400
4	Cobertura TERCEROS_LUNAS, potencia entre 90 y 110	Precio 630
5	Cobertura TODO_RIESGO, potencia mayor que 110	Precio 1200
6	Seguro dentro del primer año	Aplicar descuento del 20%

Clase de prueba

La clase creada ha sido: *SeguroTest*

Los tests implementados fueron:

- *precioDevuelveCeroSiFechaInicioEsNull*
- *precioDevuelveCeroSiFechaInicioEsFutura*
- *precioTercerosSinRecargoNiOferta*
- *precioTercerosLunasConRecargoDelCincoPorCiento*
- *precioTodoRiesgoConRecargoDelVeintePorCiento*
- *precioAplicaOfertaDurantePrimerAnio*

Resultado

Los tests de SeguroTest también pasaron correctamente con Maven.

Después comprobé la cobertura con EclEmma y el método precio() quedó cubierto en verde. Por tanto, no fue necesario añadir más casos de prueba para esta parte.

4. Pruebas de integración de VistaAgente

La segunda parte de la práctica consiste en probar la clase VistaAgente, que pertenece a la capa de presentación. En este caso no se prueba la clase de forma aislada, sino junto con el resto de capas de la aplicación: negocio y DAO.

Por eso estas pruebas son pruebas de integración.

La práctica indica que estas pruebas deben usar AssertJ Swing y que el nombre de la clase debe incluir IT, ejecutándose en la fase verify de Maven mediante maven-failsafe-plugin.

4.1 Preparación de Maven para pruebas de integración

En el módulo seguros-gui he añadido las dependencias necesarias:

- JUnit;
- AssertJ Swing.

También he configurado el plugin: *maven-failsafe-plugin*

Esto permite que las clases terminadas en IT se ejecuten durante: *mvn clean verify*

La clase de prueba creada ha sido: *VistaAgenteIT*

4.2 Funcionalidad probada

La funcionalidad probada corresponde al caso de uso: *Consulta Cliente*

La ventana VistaAgente permite introducir un DNI, pulsar el botón “Buscar” y mostrar los datos del cliente.

En el enunciado se proporciona el estado inicial de la base de datos H2. Por ejemplo, aparece el cliente con DNI 11111111A, cuyo nombre es Juan, junto con sus seguros.

4.3 Casos de prueba diseñados

Caso 1: consulta de cliente válido

Campo	**Valor**
DNI introducido	11111111A
Resultado esperado	Nombre Juan
Total esperado	1820.0

Este caso comprueba que la interfaz llama correctamente a la capa de negocio y que se muestran los datos del cliente.

Caso 2: consulta de cliente no válido

Campo	**Valor**
DNI introducido	99999999Z
Resultado esperado	Mensaje DNI No Válido
Total esperado	Campo vacío

Este caso comprueba que la aplicación responde correctamente cuando el DNI no existe.

Caso 3: comprobación de los seguros del cliente

Campo	**Valor**
DNI introducido	11111111A
Resultado esperado	Lista de seguros del cliente

Los seguros esperados fueron:

1111AAA TERCEROS
1111BBB TODO_RIESGO
1111CCC TERCEROS

Este caso comprueba que la lista de seguros se rellena correctamente en la interfaz.

4.4 Implementación de las pruebas de integración

Para abrir la ventana se usó la misma configuración que en la clase Runner:

```
IClientesDAO daoClientes = new ClientesDAO();
ISegurosDAO daoSeguros = new SegurosDAO();
```

GestionSeguros negocio = new GestionSeguros(daoClientes, daoSeguros);

VistaAgente vista = new VistaAgente(negocio, negocio, negocio);

En la prueba se utilizó FrameFixture de AssertJ Swing para interactuar con la ventana:

window.textBox("txtDNICliente")
window.button("btnBuscar")
window.textBox("txtNombreCliente")
window.textBox("txtTotalCliente")
window.list("listSeguros")

También se añadió un nombre al componente listSeguros para poder localizarlo desde la prueba.

4.5 Problemas encontrados y corrección

Al principio las pruebas de integración fallaban. El problema era que, aunque la ventana se abría, los campos no se actualizaban después de pulsar el botón.

Para comprobar si el problema estaba en la capa de negocio o en la GUI, hice una prueba temporal de backend. Esa prueba comprobó que al llamar directamente a la capa de negocio sí se obtenía correctamente el cliente 11111111A.

Por tanto, el fallo estaba en la forma en la que el test de interfaz estaba interactuando con el botón.

La solución fue modificar la prueba para ejecutar el click del botón de forma más controlada usando:

GuiActionRunner.execute(() -> window.button("btnBuscar").target().doClick());

Después de este cambio, las pruebas de integración pasaron correctamente.

4.6 Ejecución y cobertura

Después de corregir las pruebas, ejecuté: *mvn clean verify*

El resultado fue: *BUILD SUCCESS*

También ejecuté la cobertura con EclEmma sobre VistaAgenteIT usando: *Coverage As > JUnit Test*

El resultado fue que la clase VistaAgente quedó cubierta en verde, por lo que no fue necesario añadir nuevos casos de prueba.

5. Resumen de pruebas realizadas

Tipo de prueba	Clase probada	Clase de test	Herramienta
Unitaria	Cliente	ClienteTest	JUnit
Unitaria	Seguro	SeguroTest	JUnit
Integración	VistaAgente + negocio + DAO	VistaAgenteIT	JUnit + AssertJ Swing

6. Técnicas utilizadas

Caja negra

Se han usado técnicas de caja negra para diseñar casos a partir del comportamiento esperado de los métodos.

En Cliente, se han considerado casos como:

- cliente sin seguros;
- cliente con seguros;
- cliente con minusvalía;
- lista de seguros null;
- seguro null dentro de la lista.

En Seguro, se han considerado particiones según:

- fecha null;
- fecha futura;
- fecha válida;
- tipo de cobertura;
- tramos de potencia;
- primer año de seguro o no.

En VistaAgente, se han considerado entradas válidas y no válidas para el campo DNI.

Caja blanca

Después de ejecutar las pruebas de caja negra, se comprobó la cobertura con EclEmma.

En Cliente, se comprobó que el método totalSeguros() quedaba cubierto.

En Seguro, se comprobó que el método precio() quedaba cubierto.

En VistaAgente, se comprobó que la funcionalidad principal de consulta de cliente quedaba cubierta.

7. Conclusiones

En esta práctica he realizado pruebas unitarias y de integración sobre la aplicación de seguros.

Primero he probado las clases del dominio Cliente y Seguro con JUnit. En estas pruebas he comprobado los métodos que tenían lógica real, evitando los getters y setters. Después he revisado la cobertura con EclEmma y he comprobado que las partes importantes quedaban cubiertas.

Después he realizado pruebas de integración sobre VistaAgente, usando AssertJ Swing. Estas pruebas comprueban que la interfaz gráfica funciona correctamente junto con la capa de negocio y la capa DAO. Se han probado casos de consulta de cliente válido, cliente no válido y carga de seguros del cliente.

Durante la implementación de las pruebas de integración aparecieron fallos, pero sirvieron para detectar que el problema estaba en la forma de interactuar con la interfaz desde el test. Tras ajustar la prueba, la ejecución final con Maven terminó correctamente con BUILD SUCCESS.

Por tanto, considero que la práctica queda completada, ya que se han diseñado, implementado, ejecutado y revisado mediante cobertura las pruebas unitarias y de integración solicitadas.